

Teach Module 14: DTOs, Validation, and API Contracts

This README captures the Module 14 teaching notes, practice exercises, and lab. The bootcamp document was used only to identify the module topic. The teaching content below is original explanatory material.

Module Overview

Module 14 focuses on three connected ideas:

- DTOs, or Data Transfer Objects
- Validation for incoming API requests
- API contracts that clearly define request and response behavior

When you build APIs, you usually have three different shapes of data:

1. Entity: represents how data is stored in the database.
2. Request DTO: represents what the client is allowed to send.
3. Response DTO: represents what the API returns.

A common beginner mistake is exposing the database entity directly through the controller.

```
@RestController
@RequestMapping("/users")
public class UserController {

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.create(user);
    }
}
```

This works, but it creates problems. The client can send fields they should not control, such as `id`, `role`, `createdAt`, or `isAdmin`. Also, if your database model changes, your API contract changes accidentally.

A better design uses DTOs.

```
public class CreateUserRequest {
    private String name;
    private String email;
    private String password;

    // getters and setters
}
```

```
public class UserResponse {
    private Long id;
    private String name;
    private String email;
```

```
// getters and setters  
}
```

Now the controller becomes clearer:

```
@PostMapping  
public UserResponse createUser(@Valid @RequestBody CreateUserRequest request) {  
    return userService.createUser(request);  
}
```

The request object says: this is what a client may send.

The response object says: this is what the API promises to return.

That promise is the API contract.

Why DTOs Matter

DTO stands for Data Transfer Object.

A DTO is not usually where business logic lives. Its job is to carry data across a boundary, especially between:

- client and server
- controller and service
- service and external API
- backend and frontend

Think of the entity as the internal model and the DTO as the public-facing model. The client should not automatically see every field from the database.

Example entity:

```
@Entity  
public class User {  
  
    @Id  
    @GeneratedValue  
    private Long id;  
  
    private String name;  
  
    private String email;  
  
    private String passwordHash;  
  
    private String role;  
  
    private LocalDateTime createdAt;  
}
```

You should probably not return this whole object from your API because it exposes internal fields.

Instead:

```
public class UserResponse {  
    private Long id;  
    private String name;  
    private String email;  
}
```

The response hides `passwordHash` , `role` , and `createdAt` unless you intentionally include them.

Validation

Validation checks that incoming data is acceptable before it reaches business logic.

In Spring Boot, this is usually done with Bean Validation annotations.

```
public class CreateUserRequest {  
  
    @NotBlank  
    private String name;  
  
    @NotBlank  
    @Email  
    private String email;  
  
    @NotBlank  
    @Size(min = 8)  
    private String password;  
}
```

Then in the controller:

```
@PostMapping  
public UserResponse createUser(@Valid @RequestBody CreateUserRequest request) {  
    return userService.createUser(request);  
}
```

The important part is `@Valid` . Without `@Valid` , the annotations exist but are not enforced at the controller boundary.

Common validation annotations:

```
@NotNull
```

Value must not be null.

```
@NotBlank
```

String must not be null, empty, or only whitespace.

```
@Email
```

String must look like an email address.

```
@Size(min = 8, max = 100)
```

String, collection, or array size must be within range.

```
@Min(1)  
@Max(100)
```

Numeric value must be within bounds.

Request DTO vs Response DTO

Use separate DTOs for requests and responses.

Weak design:

```
public class UserDto {  
    private Long id;  
    private String name;  
    private String email;  
    private String password;  
    private String role;  
}
```

This class is trying to do too much.

Better design:

```
public class CreateUserRequest {  
    private String name;  
    private String email;  
    private String password;  
}
```

```
public class UpdateUserRequest {  
    private String name;  
    private String email;  
}
```

```
public class UserResponse {  
    private Long id;  
    private String name;  
    private String email;  
}
```

Each DTO matches one API operation. That makes the API easier to understand, test, and evolve.

Mapping DTOs to Entities

Somewhere in the application, you need to convert between DTOs and entities.

Simple manual mapping:

```
public User toEntity(CreateUserRequest request) {  
    User user = new User();  
    user.setName(request.getName());  
    user.setEmail(request.getEmail());  
    user.setPasswordHash(passwordEncoder.encode(request.getPassword()));  
    return user;  
}
```

Entity to response:

```
public UserResponse toResponse(User user) {  
    UserResponse response = new UserResponse();  
    response.setId(user.getId());  
    response.setName(user.getName());  
    response.setEmail(user.getEmail());  
    return response;  
}
```

For small projects, manual mapping is fine. For larger projects, teams often use MapStruct or similar tools.

API Contract

An API contract defines what callers can expect.

Example:

```
POST /users  
Content-Type: application/json
```

Request body:

```
{  
  "name": "Aisha Khan",  
  "email": "aisha@example.com",  
  "password": "StrongPass123"  
}
```

Successful response:

```
{  
  "id": 42,  
  "name": "Aisha Khan",  
}
```

```
"email": "aisha@example.com"
}
```

Validation error response:

```
{
  "status": 400,
  "message": "Validation failed",
  "errors": {
    "email": "must be a well-formed email address",
    "password": "size must be between 8 and 2147483647"
  }
}
```

The contract includes:

- endpoint path
- HTTP method
- request shape
- response shape
- validation rules
- status codes
- error format

Mini Exercise

Create a DTO for adding a product.

Requirements:

- `name` is required
- `price` must be greater than 0
- `description` can be optional
- `stockQuantity` cannot be negative

Possible answer:

```
public class CreateProductRequest {

    @NotBlank
    private String name;

    @Positive
    private BigDecimal price;

    private String description;

    @Min(0)
    private int stockQuantity;
}
```

Controller method:

```

@PostMapping("/products")
public ProductResponse createProduct(
    @Valid @RequestBody CreateProductRequest request
) {
    return productService.createProduct(request);
}

```

Key lesson: DTOs protect the internal model, validation protects the application, and API contracts protect clients from surprise changes.

Practice Exercises

Exercise 1: Create Request and Response DTOs

Build DTOs for a `Customer` API.

Create:

- `CreateCustomerRequest`
- `UpdateCustomerRequest`
- `CustomerResponse`

Fields:

- `id`
- `firstName`
- `lastName`
- `email`
- `phoneNumber`
- `createdAt`

Practice goal:

- `CreateCustomerRequest` should not include `id` or `createdAt`
- `UpdateCustomerRequest` should allow updating only editable fields
- `CustomerResponse` should include safe fields returned to the client

Exercise 2: Add Bean Validation

Add validation to `CreateCustomerRequest`.

Rules:

- `firstName` : required
- `lastName` : required
- `email` : required and valid email format
- `phoneNumber` : optional, but if present should have 10 digits

Example:

```

@NotBlank
private String firstName;

@NotBlank

```

```
private String lastName;

@NotBlank
@email
private String email;
```

Practice goal:

Use `@Valid` in the controller so invalid requests are rejected before reaching service logic.

Exercise 3: Product API Contract

Design an API contract for creating a product.

Endpoint:

```
POST /products
```

Request fields:

- name
- description
- price
- stockQuantity
- category

Validation rules:

- name : required
- price : must be greater than 0
- stockQuantity : cannot be negative
- category : required
- description : optional, max 500 characters

Write:

- request JSON example
- success response JSON example
- validation error JSON example
- expected HTTP status codes

Exercise 4: Prevent Entity Exposure

Given this entity:

```
@Entity
public class Employee {
    private Long id;
    private String name;
    private String email;
    private String salary;
    private String ssn;
```

```
    private String department;  
}
```

Create a safe response DTO.

Practice goal:

Do not expose sensitive fields like:

- salary
- ssn

Return only:

- id
- name
- email
- department

Exercise 5: Manual DTO Mapping

Write mapper methods:

```
Employee toEntity(CreateEmployeeRequest request)  
EmployeeResponse toResponse(Employee employee)
```

Practice goal:

Understand where data transformations happen.

Example:

```
public EmployeeResponse toResponse(Employee employee) {  
    EmployeeResponse response = new EmployeeResponse();  
    response.setId(employee.getId());  
    response.setName(employee.getName());  
    response.setEmail(employee.getEmail());  
    response.setDepartment(employee.getDepartment());  
    return response;  
}
```

Exercise 6: Validation Error Handling

Create a global validation error handler using:

```
@RestControllerAdvice
```

Handle:

```
MethodArgumentNotValidException
```

Return a clean error response like:

```
{
  "status": 400,
  "message": "Validation failed",
  "errors": {
    "email": "must be a well-formed email address",
    "name": "must not be blank"
  }
}
```

Practice goal:

Make validation errors useful for API clients.

Exercise 7: Separate Create and Update DTOs

Create two DTOs:

- CreateBookRequest
- UpdateBookRequest

Rules for create:

- title required
- author required
- isbn required
- price required and positive

Rules for update:

- title optional
- author optional
- price optional but positive if provided

Practice goal:

Understand why one generic BookDto is often weaker than operation-specific DTOs.

Exercise 8: API Contract Consistency

Design contracts for:

```
GET /orders/{id}
POST /orders
PUT /orders/{id}
DELETE /orders/{id}
```

For each endpoint, define:

- request body, if any
- response body
- success status code
- error status codes

Practice goal:

Think like an API designer, not just a Java coder.

Best Capstone-Style Practice

Build a small Spring Boot CRUD API for one domain:

- Customer
- Product
- Book
- Employee
- Order

Include:

- entity
- create request DTO
- update request DTO
- response DTO
- controller
- service
- manual mapper
- validation annotations
- global validation error response

This one exercise covers almost the entire module.

Module 14 Lab: DTOs, Validation, and API Contracts

Lab Goal

Build a Spring Boot REST API for managing Products using:

- Entity
- Request DTOs
- Response DTO
- Bean Validation
- Manual DTO mapping
- Clean validation error handling

Scenario

You are building a product catalog API. Clients should be able to create, update, and retrieve products, but they should not interact directly with your database entity.

Part 1: Create the Product Entity

Create a Product entity with:

```
private Long id;  
private String name;  
private String description;  
private BigDecimal price;  
private Integer stockQuantity;
```

```
private String category;
private LocalDateTime createdAt;
```

The `id` and `createdAt` fields should be controlled by the application, not by the client.

Part 2: Create DTOs

Create this request DTO:

```
CreateProductRequest
```

Fields:

- `name`
- `description`
- `price`
- `stockQuantity`
- `category`

Create this update DTO:

```
UpdateProductRequest
```

Fields:

- `name`
- `description`
- `price`
- `stockQuantity`
- `category`

Create this response DTO:

```
ProductResponse
```

Fields:

- `id`
- `name`
- `description`
- `price`
- `stockQuantity`
- `category`
- `createdAt`

Part 3: Add Validation

Add validation rules to `CreateProductRequest` :

- `name` : required
- `description` : optional, max 500 characters
- `price` : required and greater than 0

- `stockQuantity` : required and cannot be negative
- `category` : required

Suggested annotations:

```
@NotBlank
@Size(max = 500)
@NotNull
@Positive
@Min(0)
```

Example:

```
@NotBlank
private String name;

@Size(max = 500)
private String description;

@NotNull
@Positive
private BigDecimal price;

@NotNull
@Min(0)
private Integer stockQuantity;

@NotBlank
private String category;
```

Part 4: Create Controller Endpoints

Create a controller with these endpoints:

```
POST /products
GET /products/{id}
GET /products
PUT /products/{id}
DELETE /products/{id}
```

Use DTOs, not entities, in the controller.

Example:

```
@PostMapping
public ProductResponse createProduct(
    @Valid @RequestBody CreateProductRequest request
) {
    return productService.createProduct(request);
}
```

Part 5: Create Manual Mapper Methods

Create mapper methods like:

```
private Product toEntity(CreateProductRequest request)
```

```
private ProductResponse toResponse(Product product)
```

Practice keeping entity details separate from API details.

Part 6: Add Validation Error Handling

Create a global exception handler using:

```
@RestControllerAdvice
```

Handle:

```
MethodArgumentNotValidException
```

Return this kind of response:

```
{
  "status": 400,
  "message": "Validation failed",
  "errors": {
    "name": "must not be blank",
    "price": "must be greater than 0"
  }
}
```

Part 7: Test with Bad Requests

Send this invalid request:

```
{
  "name": "",
  "description": "Test product",
  "price": -10,
  "stockQuantity": -5,
  "category": ""
}
```

Expected result:

```
400 Bad Request
```

With field-level validation errors.

Part 8: Test with Valid Request

Send:

```
{
  "name": "Mechanical Keyboard",
  "description": "Compact keyboard with blue switches",
  "price": 79.99,
  "stockQuantity": 25,
  "category": "Electronics"
}
```

Expected response:

```
{
  "id": 1,
  "name": "Mechanical Keyboard",
  "description": "Compact keyboard with blue switches",
  "price": 79.99,
  "stockQuantity": 25,
  "category": "Electronics",
  "createdAt": "2026-08-01T10:30:00"
}
```

Completion Criteria

You are done when:

- Controllers use DTOs, not entities
- Invalid input returns `400 Bad Request`
- Validation messages are readable
- Response DTO does not expose unwanted internal fields
- Create and update operations use separate request models
- API responses are consistent across endpoints